Smaller Is Not Always Faster: The Memory–Performance Trade-off of LLM Quantization on an RTX 5090

Sijia Zhong Zheng Zheng Haofei Sun
CSE 6359 Advanced Topics in Systems & Architecture, Fall 2026
The University of Texas at Arlington

***Abstract*—Quantization is the standard way to fit large language models into GPU memory, but a smaller model is not automatically a faster one. We measure the memory–performance trade-off end to end on an NVIDIA RTX 5090 for Qwen2.5 models with 3B, 7B and 14B parameters in eight configurations: BF16, LLM.int8() and NF4 on Hugging Face Transformers, and BF16, FP8 W8A8, GPTQ-INT8, AWQ-INT4 and GPTQ-INT4 on vLLM, and repeat the core measurements on Qwen3-8B and, at 4 bits, Qwen3-32B. The study covers 1,890 timed runs, a replay of 300 ShareGPT requests, WikiText-2 perplexity, and GSM8K and MMLU accuracy with paired significance tests. Real weight savings fall short of the nominal ratio (2.60× rather than 4× for 4-bit weights on the 7B model) and shrink further at run time because the KV cache and activations stay in 16 bits. Speed depends on the kernel and the regime rather than on the bit-width: at batch size one, 8-bit weights decode 1.64× faster than BF16 with vLLM's Marlin kernel but 3.4× slower with bitsandbytes LLM.int8(), and 4-bit AWQ weights decode 2.44× faster. Weight-only formats stop paying off once batching makes decoding compute-bound, whereas FP8 W8A8 stays faster everywhere and delivers 1.73× the BF16 throughput at batch size 256. Kernel microbenchmarks, a measured roofline and per-step profiles explain every crossover. Quality depends on the bit-width and the model size: GPTQ-INT8 and FP8 stay within 0.9 points of BF16 on GSM8K and MMLU, whereas 4-bit formats cost 0.6–2.8 MMLU points, most on the smallest model; still, a 14B model in AWQ-INT4 (10.1 GB) beats the 7B model in BF16 (15.2 GB) on both benchmarks. We release the harness, the raw measurements and a decision guide for practitioners.**

I. Introduction

Language models keep growing, from GPT-2's 1.5B parameters (2019) to GPT-3's 175B (2020) and DeepSeek-V3's 671B (2024). Memory is the first wall a deployment hits: the BF16 weights of a 7B-parameter model alone take 15 GB, and the KV cache of every concurrent request comes on top. Quantization stores the weights (and sometimes the activations) in 8 or 4 bits, and is usually advertised with its nominal ratio: 2× smaller for INT8, 4× for INT4.

Smaller is not automatically faster. Low-bit weights must be dequantized before the tensor cores can use them. A speedup exists only if a kernel hides that work, and the bytes saved matter only when the GPU is waiting on memory rather than on arithmetic. Whether quantization pays off therefore depends on the batch size, the prompt length and above all the software that runs it. This project measures that trade-off end to end on a current consumer GPU, an NVIDIA RTX 5090, and answers four questions:

- **Q1 (memory).** How much does quantization reduce the actual inference-time GPU memory, compared with its theoretical weight compression?
- **Q2 (speed).** How do TTFT, decode latency (TPOT) and throughput change?
- **Q3 (crossover).** At which batch sizes and prompt lengths does a lower-bit format become faster—or slower—than BF16?
- **Q4 (quality).** How much perplexity and task accuracy does it cost?

Contributions. (1) A reproducible measurement harness (`quantbench`) that records memory, latency, throughput, energy and quality for eight configurations on two backends (Hugging Face Transformers with bitsandbytes, and vLLM with Marlin/CUTLASS kernels), three model sizes and a second model family (Qwen3-8B, and Qwen3-32B, which fits the GPU only at 4 bits), about 1,890 timed runs in total. (2) Evidence that the kernel matters more than the bit-width: at batch size 1 the 7B model's 8-bit weights decode 1.64× faster than BF16 under vLLM's Marlin kernel but 3.4× slower under bitsandbytes LLM.int8(), and its 4-bit weights 2.44× faster under Marlin but only 1.13× faster under bitsandbytes NF4. (3) A kernel-level explanation of the crossover from microbenchmarks, a measured roofline and per-step profiles, which locates the batch size where each format stops paying off. (4) Memory and quality results on three benchmarks with paired significance tests, showing that the nominal ratio overstates the real saving, that 8-bit formats with good kernels preserve quality while 4-bit formats cost more the smaller the model, and that a quantized larger model still beats a full-precision smaller one at less memory. We also show that a single quality metric can miss quantization damage. We end with a decision guide (Section V). Code, raw measurements and figures are public.

II. Background

A. Two phases of LLM inference

Autoregressive generation runs in two phases with very different hardware behavior. In *prefill*, all L prompt tokens of each of the B requests pass through the network at once, so every linear layer is a GEMM with $M = B \cdot L$ rows. Its cost determines the *time to first token* (TTFT). In *decode*, every

step produces one new token per request, so the same weights are multiplied by a matrix with only $M = B$ rows. Its cost per step is the *time per output token* (TPOT). The keys and values of all previous tokens are kept in a KV cache whose size grows with $B \cdot (L + N)$ for N generated tokens.

B. A roofline view of quantization

For a linear layer $y = xW^\top$ with $x \in \mathbb{R}^{M\times K}$ and $W \in \mathbb{R}^{N\times K}$ stored with b bytes per weight, the arithmetic intensity is

$$I(M) \;=\; \frac{2MNK}{b\,NK + 2MK + 2MN} \;\approx\; \frac{2M}{b} \quad (M \ll N, K). \tag{1}$$

A kernel is memory-bound while $I(M)$ is below the GPU's ridge point P_{peak}/BW and compute-bound above it [1]. With BF16 weights ($b = 2$), decode at small B is deeply memory-bound: a step costs roughly the time needed to read every weight once. Storing weights in 4 bits ($b \approx 0.5$) cuts these bytes by about $4\times$, which is the source of every decode-time speedup reported for weight-only quantization. Three things work against it. (i) The weights must be dequantized back to 16 bits before the tensor cores can use them, and this extra work is free only if a fused kernel hides it under the memory stream. (ii) Weight-only formats (W4A16, W8A16) still compute in BF16, so once $I(M)$ reaches the ridge, at a $4\times$ smaller M than BF16, the kernel is compute-bound and the dequantization overhead is no longer hidden. (iii) Prefill is compute-bound from the start, so weight-only formats cannot shorten TTFT. Formats that quantize the activations too (W8A8 in INT8 or FP8 [2]) also raise the compute ceiling, because the tensor cores run 8-bit math at twice the BF16 rate.

C. Quantization methods studied

LLM.int8() [3] quantizes weights per output row and activations per token to INT8. Activation feature columns that contain any value above a threshold (6.0) are treated as outliers and multiplied in FP16 (mixed-precision decomposition). **NF4** [4] stores weights in the 4-bit NormalFloat format with one FP32 absmax scale per 64-weight block. Both are implemented in bitsandbytes and applied to the BF16 checkpoint at load time through Hugging Face Transformers [5]. **GPTQ** [6] uses approximate second-order information to choose rounding that minimizes each layer's output error. **AWQ** [7] instead rescales the weight channels that matter most for the activations before rounding. Both produce INT4 (or INT8) weights with one FP16 scale per group of 128 weights, and are executed in vLLM [8] by the Marlin mixed-precision GEMM kernels [9]. **FP8 W8A8** casts the weights to E4M3 at load time with one scale per tensor and quantizes the activations dynamically per token; vLLM runs it with CUTLASS FP8 GEMMs. SmoothQuant [10] motivates the W8A8 design by moving activation outliers into the weights; we use FP8 as its hardware-native counterpart on Blackwell.

TABLE I
CONFIGURATIONS. W/A: WEIGHT/ACTIVATION PRECISION DURING THE GEMM.

Backend	Name	W/A	Method	Kernel	Source
HF	BF16	16/16	—	cuBLAS	BF16 ckpt
HF	INT8	8/8†	LLM.int8()	bnb/cuBLASLt	on load
HF	NF4	4/16	NF4, block 64	bnb dequant/GEMV	on load
vLLM	BF16	16/16	—	cuBLAS	BF16 ckpt
vLLM	FP8	8/8	FP8 E4M3	CUTLASS	on load
vLLM	GPTQ-8	8/16	GPTQ g128	Marlin	Qwen ckpt
vLLM	AWQ-4	4/16	AWQ g128	Marlin	Qwen ckpt
vLLM	GPTQ-4	4/16	GPTQ g128	Marlin	Qwen ckpt

† Activation columns with outliers ($|x| > 6$) are multiplied in FP16.

III. METHODOLOGY

A. Platform

All measurements run on one NVIDIA GeForce RTX 5090 (Blackwell GB202, compute capability 12.0, 170 SMs, 32 GB GDDR7 on a 512-bit bus, 96 MB L2, 575 W board power, PCIe 5.0 x16) in a workstation with an Intel Core Ultra 9 285K and 64 GB of RAM, running Ubuntu 24.04, driver 595.91, PyTorch 2.13 (CUDA 13.0 runtime), Transformers 5.19, bitsandbytes 0.50.2 and vLLM 0.32 (V1 engine, FlashAttention backend, CUDA graphs and `torch.compile` enabled). The GPU's practical ceilings, measured with our own microbenchmarks (Section IV-D), are 1.61 TB/s of DRAM bandwidth and 218 TFLOP/s of dense BF16 GEMM throughput, which puts the BF16 ridge point at about 135 FLOP/byte.

B. Models and configurations

We use the instruction-tuned Qwen2.5 models [11] at 3B, 7B and 14B parameters. Table I lists the eight configurations. The Hugging Face (HF) backend is what a practitioner gets from one extra argument to `from_pretrained`: the BF16 checkpoint is quantized on the fly by bitsandbytes and executed eagerly. The vLLM backend is a production serving engine that runs the official Qwen GPTQ and AWQ checkpoints (group size 128) with Marlin kernels and quantizes the BF16 checkpoint to FP8 at load time. In every configuration the token embedding, the LM head and the norms stay in 16-bit.

C. Workloads

Controlled sweep. Each configuration generates exactly $N = 128$ tokens (EOS suppressed) for $B \in \{1, 2, 4, \ldots, 64\}$ prompts (up to 256 on vLLM) of $L \in \{128, 512, 1024, 2048\}$ random tokens. Content does not change the speed of a dense model, and random prompts make every precision do identical work. A smaller sweep varies $N \in \{32, 128, 512\}$. **Real workload.** We draw 300 conversations from ShareGPT [12] with a fixed seed and the same filters as vLLM's benchmark (prompt $\leq$1024 tokens, prompt+output $\leq$2048). The first user turn is the prompt and the first assistant turn sets the exact output length (median prompt 31 tokens, median output 284 tokens). vLLM replays them through its OpenAI-compatible server with `vllm bench serve` at 1, 4, 16 and 64 concurrent clients. HF processes them in arrival order in static batches of

16. **Quality.** WikiText-2 [13] perplexity uses the GPTQ/AWQ protocol: non-overlapping 2048-token windows of the raw test split. GSM8K [14] accuracy uses all 1,319 test questions with zero-shot chain-of-thought (Qwen's "reason step by step, answer in `\boxed{}`" instruction) and greedy decoding up to 512 tokens. MMLU [15] accuracy uses all 14,042 test questions zero-shot; the prediction is the option letter the model scores highest as the first token of its answer. Instruct models are prompted through their chat template; Qwen3 runs in non-thinking mode.

D. Metrics and protocol

On HF, a streamer timestamps every token that `generate` emits; each emission copies the token to the host, so a timestamp is taken only after the GPU has produced that token. TTFT is the time to the first emission, TPOT $=$ (E2E$-$TTFT)$/(N-1)$, and throughput is BN/E2E. On vLLM we follow `vllm bench latency`: TTFT is the latency of the batch with one output token, and TPOT is the extra latency of N tokens divided by $N-1$. Prefix caching is disabled so repeats never reuse KV blocks. Every cell runs one warm-up and three measured repetitions with fresh prompts; the coefficient of variation stays below 0.7%. Memory comes from PyTorch's allocator (peak allocated) and from NVML (device total). Energy is the difference of NVML's cumulative energy counter over the three repetitions, divided by the number of generated tokens. The GPU also drives the workstation's display, so we report means over repetitions and do not lock clocks; the average SM clock during the measurements is 2812 MHz.

IV. Results

A. Q1: how much memory does quantization actually save?

Weights. Figure 1 and Table VI compare three numbers per configuration: the naive estimate, our analytic model (only the decoder's linear layers are quantized; embedding, LM head, norms and biases stay in 16 bits; each format stores its scales), and the memory measured on the GPU. The model predicts every HF measurement within 0.6%, whereas the naive estimate under-predicts NF4 by up to 57%. The 16-bit leftovers are large: the embedding and LM head hold 14% of Qwen2.5-7B's parameters, so NF4 shrinks the weights 2.60× instead of 4×, INT8 1.75× instead of 2×, and vLLM's AWQ checkpoint 2.70×.

Run time. Quantization leaves the KV cache and the activations in 16 bits, so the saving shrinks as more tokens are in flight (Figure 3). For Qwen2.5-7B on HF, NF4 cuts peak memory 2.53× for one short request but only 1.49× at $B = 32$, $L = 2048$. INT8 is worse: LLM.int8() keeps FP16 and INT8 copies of the activations plus the outlier columns, so its activation workspace is 1.6× that of BF16 and its saving drops to 1.04×. Nor did quantization let HF serve more long prompts on the 7B model: BF16, INT8 and NF4 all run out of memory at $B = 64$, $L = 2048$, where prefill activations rather than weights dominate. For the 14B model, whose BF16 weights alone take 29.5 GB, NF4 does buy capacity: it runs 16 prompts of 2048 tokens and 64 prompts of 512 tokens, which run out of memory in BF16.

Capacity. vLLM reserves a fixed share of the GPU and turns everything that the weights do not use into KV cache, so every gigabyte saved becomes cache (Figure 4). With AWQ-INT4, Qwen2.5-7B holds 1.90× as many cached tokens as with BF16 (355K vs. 187K). In BF16 the 14B model's 29.5 GB of weights leave no room for any KV cache even when vLLM may use 95% of the GPU, so it cannot be served at all, while its AWQ-INT4 version runs with 82K cached tokens. Finally, on-the-fly quantization costs load time: bitsandbytes INT8 needs 12.5 s to load the 7B model, 7× longer than BF16.

B. Q2: latency and throughput

Figure 2 and Table VII report the 7B model; the 3B model behaves alike, with one exception discussed below. **Prefill** is compute-bound, so weight-only formats cannot shorten it: AWQ and GPTQ change TTFT by at most 7%, and NF4 makes it 1.26× slower because bitsandbytes dequantizes every weight matrix before the GEMM. Formats that also quantize the activations do help: vLLM FP8 cuts the TTFT of a 2048-token prompt 1.77×, and even LLM.int8()'s INT8 GEMMs win 1.08× once a batch of long prompts amortizes their overheads.

Decode. At $B = 1$, vLLM's BF16 step takes 9.52 ms, within 99% of the time needed to stream 15.2 GB of weights at the measured bandwidth, so fewer bytes translate almost directly into speed: AWQ-INT4 is 2.44× faster, GPTQ-INT8 1.64× and FP8 1.59×. On HF the same idea fails: LLM.int8() is 3.4× slower, and NF4 is 1.13× faster on the 7B model and 1.22× on the 14B model but 1.39× slower on the 3B model, whose BF16 step is already limited by the host rather than by memory (Section IV-D).

Throughput. The decode advantage of weight-only formats fades with batch size. At $B = 256$, vLLM BF16 produces 2,766 tokens/s and AWQ-INT4 2,720—no gain from a 4× smaller weight format—while FP8 reaches 4,794, 1.73× the BF16 rate.

C. Q3: when does lower precision pay off?

Figure 5 maps the end-to-end speedup of a request batch (prompt plus 128 generated tokens) over the whole grid, and Table II locates the crossovers. Three regimes emerge. *Weight-only formats with fused kernels* (AWQ, GPTQ) win by up to 2.44× at small batches and short prompts, where decode dominates and is memory-bound; the gain shrinks with batch size and prompt length and vanishes around $B \approx 175$ ($L = 512$). *W8A8 FP8* wins everywhere (1.59×–1.80×), because it halves the bytes *and* doubles the tensor-core rate, so it also speeds up the compute-bound regime. *bitsandbytes* formats rarely win: LLM.int8() is slower in every cell (0.27×–0.71×), and NF4 wins only for the 7B model below $B \approx 7$.

Output length. The answer length decides how much of a request is decode, so it moves the crossover too (Figure 7). At $B = 16$ and 512-token prompts, AWQ-INT4 speeds up a request that generates 32 tokens by 1.26× but one

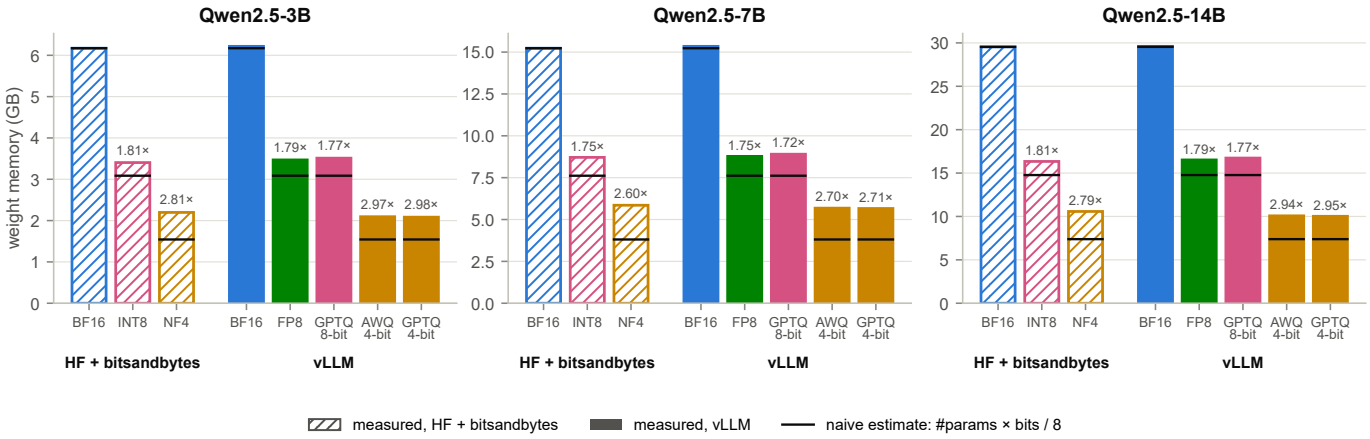


Fig. 1. Q1 — weight memory measured right after loading (bars), against the naive estimate #params × bits/8 (black ticks). Labels give the measured saving relative to BF16 on the same backend. Hatched: HF + bitsandbytes; solid: vLLM.

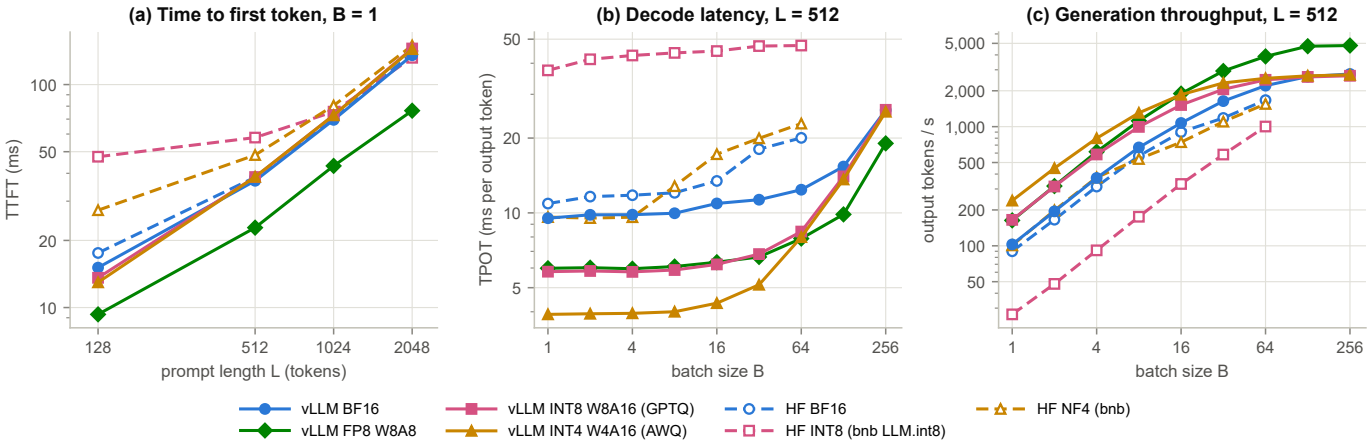


Fig. 2. Q2 — Qwen2.5-7B, 128 generated tokens. (a) TTFT for one request; (b) TPOT and (c) generation throughput for prompts of 512 tokens. Solid lines: vLLM; dashed with hollow markers: HF + bitsandbytes.

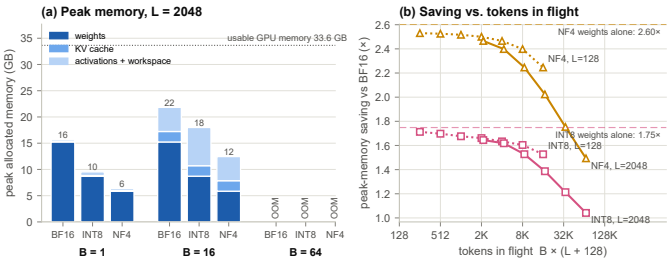


Fig. 3. Q1 — (a) peak memory of Qwen2.5-7B on HF split into weights, KV cache and activations; (b) peak-memory saving relative to BF16 as the number of tokens in flight grows (dashed: the saving of the weights alone).

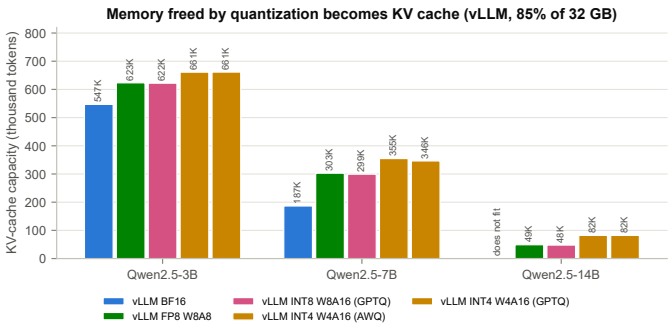


Fig. 4. Q1 — KV-cache capacity vLLM allocates with 85% of the GPU after loading the weights.

that generates 512 tokens by 2.16× (GPTQ-INT8: 1.15× to 1.61×). FP8 W8A8, which accelerates prefill and decode alike, stays near 1.72× for every length, and LLM.int8(), whose overhead sits in the decode loop, falls from 0.52× to 0.33×. Weight-only formats therefore suit chat-style workloads with long answers; short-answer workloads such as classification gain little from them.

D. *Why: kernels, rooflines and host overhead*

Kernels. Figure 6a isolates the linear layers. Marlin W4A16 is 3.2× faster than cuBLAS at $M = 1$ and keeps more than a 5% advantage until $M \approx 333$; Marlin W8A16 starts at 1.9× and loses its edge at $M \approx 164$. These are the regimes predicted

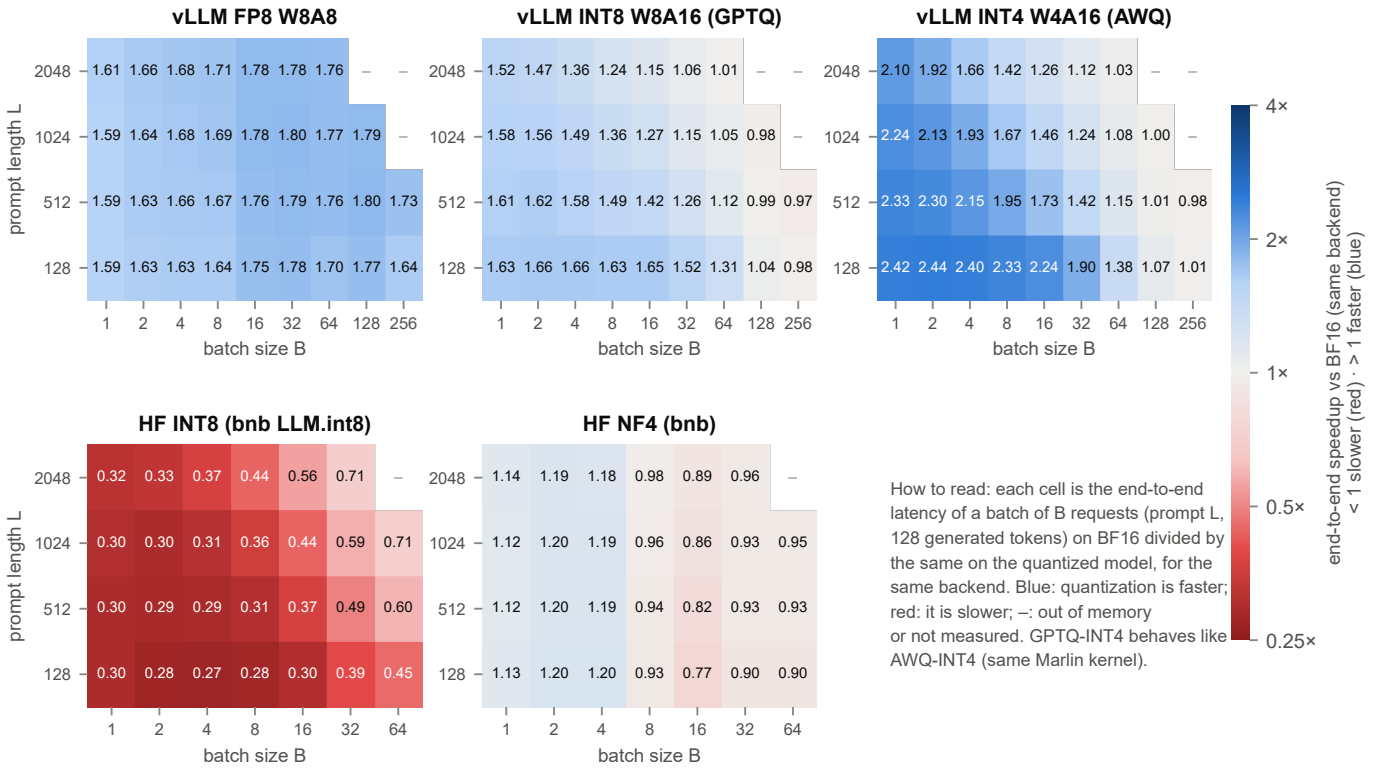


Fig. 5. Q3 — end-to-end speedup of each quantized configuration over BF16 on the same backend (Qwen2.5-7B, 128 generated tokens), over batch size and prompt length.

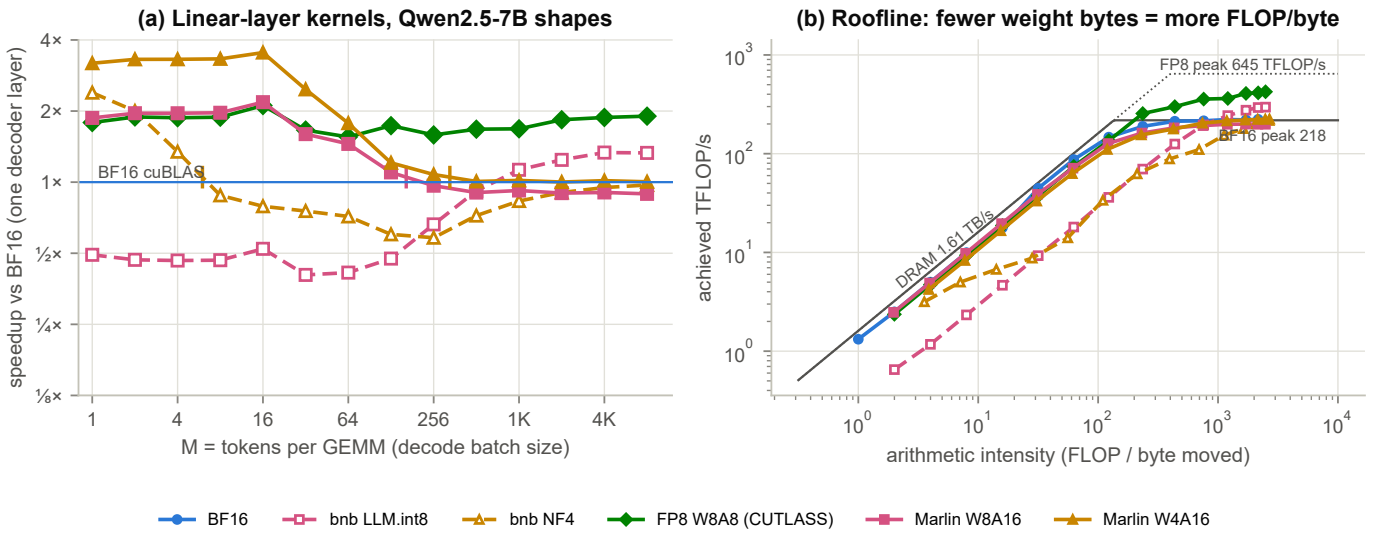


Fig. 6. Mechanism — (a) speedup of each low-bit linear-layer kernel over BF16 cuBLAS for the four GEMMs of one Qwen2.5-7B decoder layer, as a function of the number of tokens M in the batch; vertical ticks mark crossovers. (b) The same kernels on a roofline built from the measured DRAM bandwidth and GEMM peaks.

by Equation (1): fewer bytes per weight raise the arithmetic intensity, so a weight-only kernel climbs the bandwidth roof faster but reaches the compute roof at a smaller M, after which the math still runs in BF16 and dequantization is no longer hidden (Figure 6b). FP8 W8A8 is 1.8× faster at $M = 1$ and still 1.9× at $M = 8192$ because it also doubles the tensor-core rate. The bitsandbytes kernels are not slow per se: NF4's dedicated GEMV kernel is 2.39× faster at $M = 1$, but beyond a handful of tokens NF4 dequantizes whole matrices and drops to 0.58×; LLM.int8() ranges over 0.40–1.33×, winning only for prefill-sized GEMMs ($M \geq 1024$), where its INT8 tensor-core math pays for the outlier handling. That NF4 kernels are

TABLE II

Q3 — SPEEDUP OVER BF16 ON THE SAME BACKEND AT BATCH SIZE 1 AND AT THE LARGEST BATCH MEASURED, AND THE BATCH SIZE AT WHICH THE QUANTIZED CONFIGURATION STOPS WINNING (QWEN2.5-7B, $L = 512$, 128 OUTPUT TOKENS; LINEAR INTERPOLATION IN $\log B$).

Config	Decode B=1	Decode max B	Decode wins up to	E2E B=1	E2E max B	E2E wins up to
HF INT8 (LLM.int8)	0.29×	0.43× (64)	never	0.30×	0.60× (64)	never
HF NF4	1.13×	0.88× (64)	$B \approx 7$	1.12×	0.93× (64)	$B \approx 7$
vLLM FP8 W8A8	1.59×	1.36× (256)	all B	1.59×	1.73× (256)	all B
vLLM GPTQ W8A16	1.64×	1.00× (256)	$B \approx 251$	1.61×	0.97× (256)	$B \approx 123$
vLLM AWQ W4A16	2.44×	1.02× (256)	all B	2.33×	0.98× (256)	$B \approx 175$
vLLM GPTQ W4A16	2.45×	1.05× (256)	all B	2.35×	1.03× (256)	all B

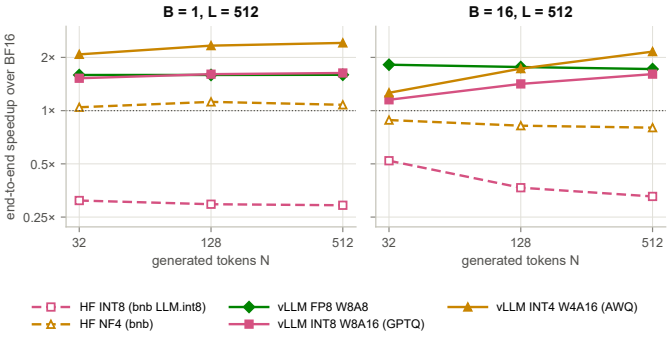


Fig. 7. Q3 — end-to-end speedup over BF16 against the number of generated tokens (Qwen2.5-7B, 512-token prompts). Longer answers raise the decode share and with it the gain of weight-only formats.

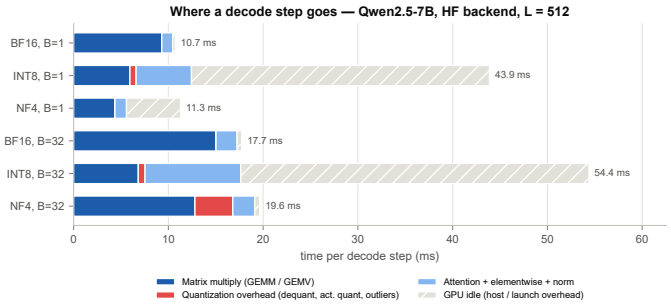


Fig. 8. Mechanism — GPU time of one HF decode step of Qwen2.5-7B ($L = 512$) by kernel category, and the time the GPU waits for the host.

fast at $M = 1$ while the NF4 model is only 1.13× faster end to end points to costs outside the GEMMs.

Host overhead. Kernels explain vLLM; the HF backend is dominated by something else. Figure 8 profiles one decode step of the 7B model. With BF16, the GPU is busy 97% of a $B = 1$ step. LLM.int8() launches 3.7× as many kernels (casts, outlier gathering, quantization and two GEMMs per layer), and the GPU is busy only 28% of the time: the step is bound by Python and kernel-launch overhead, not by the GPU. NF4 sits in between: its GEMV kernels finish quickly, but the GPU is busy only 49% of the step, which is why its 2.39× kernel speedup shrinks to 1.13× end to end. This is also why the INT8 step of the 3B model (36 layers) is 1.52× slower than that of the 7B model (28 layers).

TABLE III

REAL WORKLOAD — 300 SHAREGPT REQUESTS REPLAYED AGAINST QWEN2.5-7B (REFERENCE OUTPUT LENGTHS). C: CONCURRENT CLOSED-LOOP CLIENTS FOR VLLM, STATIC BATCH SIZE FOR HF. THROUGHPUT IN REQUESTS/S AND OUTPUT TOKENS/S; TTFT AND TPOT IN MS.

Config	C	req/s	tok/s	TTFT p50 / p99	TPOT p50 / p99
HF BF16 (static)	16	0.84	254	797 / 1244	23.5 / 29.7
HF INT8 (static)	16	0.39	119	691 / 1158	54.9 / 64.1
HF NF4 (static)	16	0.72	215	842 / 1271	27.8 / 35.4
vLLM BF16	1	0.35	104	16 / 62	9.6 / 9.6
vLLM BF16	64	11.76	3537	46 / 1654	13.6 / 16.7
vLLM FP8	1	0.54	164	11 / 44	6.1 / 6.2
vLLM FP8	64	17.92	5388	33 / 1485	8.9 / 15.1
vLLM GPTQ-8	1	0.55	166	11 / 63	6.0 / 6.2
vLLM GPTQ-8	64	15.74	4733	41 / 1743	10.5 / 18.0
vLLM AWQ-4	1	0.80	240	11 / 65	4.1 / 4.5
vLLM AWQ-4	64	17.23	5180	40 / 1747	10.1 / 14.8
vLLM GPTQ-4	1	0.80	241	11 / 62	4.1 / 4.4
vLLM GPTQ-4	64	17.23	5181	40 / 1722	10.0 / 19.3

E. Real workload: ShareGPT

Replaying 300 ShareGPT requests (Figure 9 and Table III; median prompt 31 tokens, median answer 284 tokens) confirms the controlled sweep under a realistic, decode-heavy mix. With one client the median TPOT falls from 9.6 ms with BF16 to 4.1 ms with AWQ-INT4 and 6.1 ms with FP8. As clients are added, continuous batching [16] raises the throughput of every format, and the ranking shifts toward FP8: at 64 clients FP8 serves 5,388 output tokens/s against 5,180 for AWQ-INT4 and 3,537 for BF16. The serving stack matters even more than the number format: with the same 16 requests in flight, vLLM BF16 delivers 5.3× the throughput of HF BF16 with static batching, and HF's quantized configurations are slower still (215 tokens/s for NF4 and 119 for LLM.int8()). Static batching pads every batch to its longest prompt and holds it until its longest answer is done, so its median TTFT is 797 ms against 34 ms with continuous batching.

F. Q4: quality

Table IV and Figure 10 measure quality three ways: WikiText-2 perplexity (language modeling), GSM8K (multi-step math reasoning, 1,319 questions) and MMLU (knowledge across 57 subjects, 14,042 questions). Every accuracy comparison is paired: all configurations answer the same questions, so we test each difference against HF BF16 with an exact McNemar test on the questions where exactly one of the two

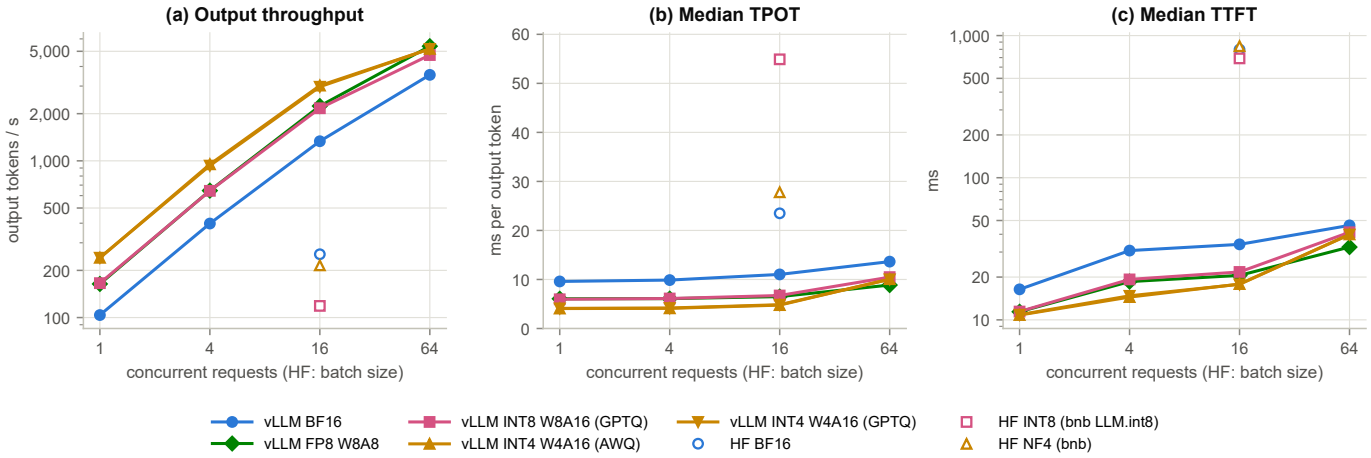


Fig. 9. Real workload — 300 ShareGPT requests replayed against Qwen2.5-7B with their reference output lengths: vLLM served at 1–64 concurrent clients (lines), HF with static batches of 16 (hollow markers). TTFT and TPOT are medians over requests.

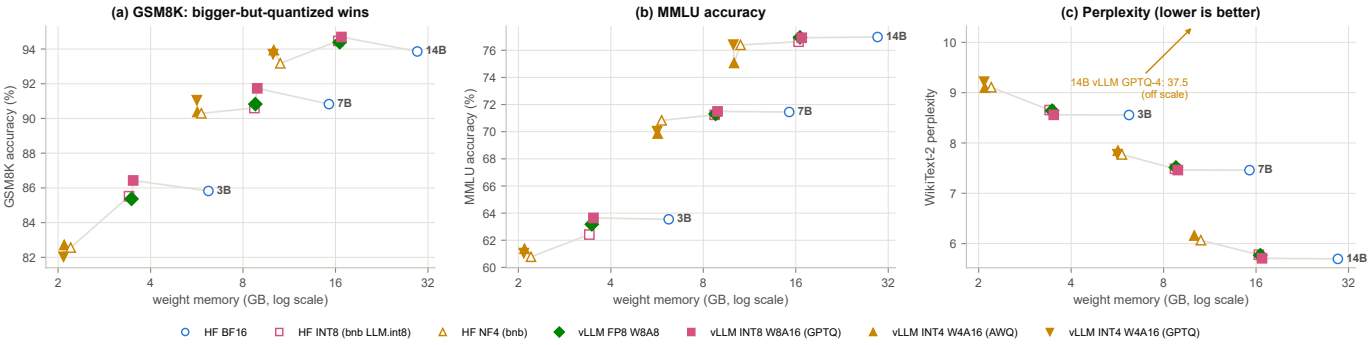


Fig. 10. Q4 — (a) GSM8K accuracy, (b) MMLU accuracy and (c) WikiText-2 perplexity of every Qwen2.5 configuration against its weight memory; lines connect the configurations of one model size. Hollow markers: HF + bitsandbytes; filled: vLLM.

runs is correct, and correct for the number of comparisons (Bonferroni) where we claim significance.

Noise floor. Even two BF16 runs of the same weights disagree. vLLM and HF use different kernels and accumulation orders, and greedy decoding turns the resulting rounding differences into different reasoning chains: the two backends disagree on 19 of the 7B model's GSM8K questions and their accuracies differ by 0.4 points. On MMLU, where the answer is a single token, they differ by less than 0.1 points (0.1 on 3B). Differences of this size are not caused by quantization.

8-bit formats preserve quality. GPTQ-INT8 and FP8 W8A8 stay within 0.9 points of BF16 on both benchmarks for every model size and change perplexity by at most 0.05 on the 7B model. On GSM8K, the only nominally significant difference is an improvement (GPTQ-INT8 on 7B, $+0.9$ points, $p = 0.043$), which does not survive a Bonferroni correction over the 12 comparisons. The one measurable 8-bit cost is LLM.int8() on the 3B model, which changes MMLU accuracy by -1.1 points ($p < 0.001$; on 7B -0.2, $p = 0.13$).

4-bit formats cost measurable quality, most on the smallest model. MMLU resolves what GSM8K cannot. The 4-bit formats change MMLU accuracy by -2.8 to -2.2 points on the 3B model, -1.6 to -0.6 on 7B and -1.9 to -0.6 on 14B; 3 of 3, 3 of 3 and 1 of 3 of these drops remain significant after correcting for all 18 MMLU comparisons. Perplexity tells the same story ($+0.31$ to $+0.39$ on 7B, $+0.54$ to $+0.66$ on 3B). On GSM8K, the 3B model's accuracy changes by -3.8 to -3.1 points (3 of 3 significant after correction), while the 7B model's changes of -0.5 to $+0.2$ points are within the noise of 1,319 questions. A smaller model has less redundancy to absorb the rounding error.

Model size versus bit-width. Figure 10 places every configuration by its weight memory. Qwen2.5-14B in AWQ-INT4 needs 10.1 GB of weights, less than the 15.2 GB of Qwen2.5-7B in BF16, and scores 93.9% vs. 90.8% on GSM8K and 75.1% vs. 71.4% on MMLU (McNemar $p < 0.001$ and $p < 0.001$). At a fixed memory budget, parameters are worth more than bits.

One metric is not enough. The 14B GPTQ-INT4 checkpoint scores 93.7% on GSM8K and changes MMLU accuracy by only -0.6 points, on par with the other 14B formats, yet its WikiText-2 perplexity is 37.5 against 5.71–6.16 for AWQ-INT4, GPTQ-INT8 and FP8. The outlier is not an artifact of our setup: it persists with BF16 instead of FP16 activations, the loss is elevated at every position of the 2048-token window, and the same harness scores the 7B GPTQ-INT4 checkpoint

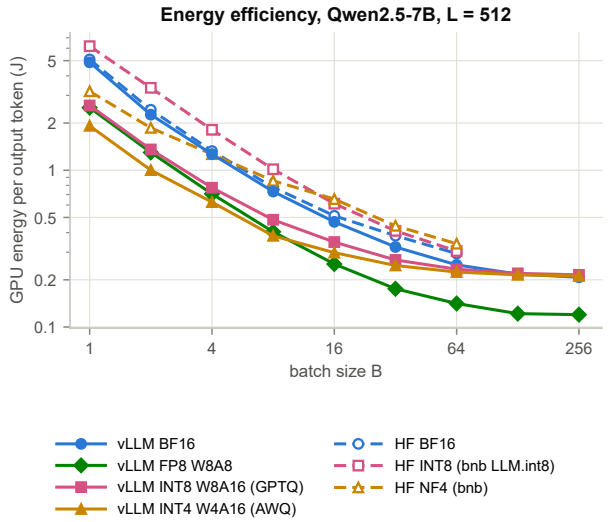


Fig. 11. GPU energy per generated token (NVML energy counter), Qwen2.5-7B, $L = 512$.

normally. Ranking the next token shows what happened. On raw WikiText the checkpoint predicts the next token correctly only 37% of the time (60–62% for the other 14B formats), whereas on chat-formatted GSM8K solutions it matches them (77% vs. 77%). The quantized weights still model the chat distribution they serve but have lost much of their general language modeling. The chat benchmarks alone would have missed this; perplexity alone would have rejected a checkpoint that works for chat.

G. Energy

Because decode power is close to the board limit in every configuration (410–579 W), energy per token follows latency (Figure 11). At $B = 1$, AWQ-INT4 needs 1.9 J per token against 4.9 J for BF16 on vLLM, 2.5× less; at $B = 256$ the GPU is power-capped at 575 W for every format and only FP8 still saves energy (0.12 vs. 0.21 J). LLM.int8() draws little power (164 W) but runs so long that it costs the most energy per token.

H. Does it generalize? Qwen3-8B

To check that the conclusions are not specific to Qwen2.5, we repeated the core measurements on Qwen3-8B [17], a newer model with a different layout (36 layers and 8 KV heads against 28 and 4 for Qwen2.5-7B), run in non-thinking mode. Table V compares the two on the same grid.

The systems results carry over almost unchanged. 4-bit weights shrink by 2.67× rather than 4×; at batch size one AWQ-INT4 decodes 2.33× faster (2.44× for Qwen2.5-7B), FP8 1.54× faster, and LLM.int8() runs at 0.28× of the BF16 speed. At $B = 64$ FP8 still raises throughput by 1.67× while AWQ-INT4 has fallen to 1.12×. The one difference confirms the host-overhead explanation of Section IV-D: NF4, 1.13× faster than BF16 on the 28-layer Qwen2.5-7B, is 0.90× on the 36-layer Qwen3-8B, just as it was slower on the 36-layer Qwen2.5-3B. More layers mean more kernel launches per decode step, and the step becomes bound by the host rather than by memory. Qwen3-8B also stores 2.6× as many KV-cache bytes per token, so at $B = 256$ none of its configurations fits the cache; the memory a format saves on weights matters even more for such models. Quality costs are smaller: Qwen3-8B's MMLU accuracy changes by -0.7 (FP8), $+0.7$ (LLM.int8()), -0.7 (AWQ-INT4), -0.3 (NF4) points relative to BF16, against -0.1 (FP8), -0.2 (LLM.int8()), -1.6 (AWQ-INT4), -0.6 (NF4) for Qwen2.5-7B, and no GSM8K change exceeds 0.6 points. The ordering seen on Qwen2.5, where 8-bit formats are free and 4-bit formats are not, does not hold here: FP8 costs as much as AWQ-INT4. Which format preserves quality is a property of the model and must be measured, not assumed from the bit-width.

Scaling up: Qwen3-32B. At 32B parameters the question changes from speed to feasibility. Qwen3-32B needs 66 GB in BF16 and about half of that in FP8 or INT8, more than the GPU's 32 GB, so on this GPU it runs only at 4 bits: AWQ-INT4 takes 19.6 GB and leaves room for 29K cached tokens, and NF4 takes 20.7 GB. Against Qwen3-8B in BF16 (16.4 GB, a similar budget), the 4-bit 32B model needs 13.3 ms per decoded token at batch size one (vLLM; 8B BF16: 10.3 ms) and scores 94.2% vs. 91.4% on GSM8K ($p < 0.001$) and 78.1% vs. 69.6% on MMLU ($p < 0.001$). A model four times larger, in 1.20× the weight memory, is significantly better on both benchmarks; on this GPU, that model exists only because of quantization. One caveat is numerical: the AWQ checkpoint ships FP16 activations, and in 13 of 146 perplexity windows the activations overflow beyond the first 512 positions and the loss becomes NaN; with BF16 activations every window is finite (perplexity 7.88), so long-context use of this checkpoint should run in BF16.

V. Discussion

A. When does quantization pay off?

Our measurements reduce to five rules for deploying a quantized LLM on a GPU like the RTX 5090.

(1) Budget memory with the real ratio, not $16/b$. Every format saves memory, but the 16-bit embedding, LM head and scales cap 4-bit formats at 2.60×–2.70× for a 7B model, and the KV cache and activations are not quantized at all. A capacity plan that assumes 4× will run out of memory.

(2) For interactive, low-concurrency serving, use 4-bit weight-only formats with fused kernels. At one request per GPU, AWQ-INT4 with Marlin decodes 2.44× faster than BF16 and uses 2.5× less energy per token.

(3) For high-throughput serving, use W8A8 FP8. Weight-only formats lose their advantage once the batch makes decode compute-bound ($B \gtrsim 175$ for AWQ at 512-token prompts) and never speed up prefill. FP8 is the only format that keeps winning, with 1.73× the BF16 throughput at $B = 256$, because it also doubles the tensor-core rate.

(4) Quantize for memory, not speed, when only bitsandbytes is available. LLM.int8() is 3.4× slower than BF16 at batch size one and never catches up, and NF4 is faster only for

TABLE IV

Q4 — QUALITY. WIKITEXT-2 PERPLEXITY (2048-TOKEN WINDOWS), GSM8K ACCURACY (%, 1,319 QUESTIONS, ZERO-SHOT COT, GREEDY) AND MMLU ACCURACY (%, 14,042 QUESTIONS, ZERO-SHOT). DELTAS ARE AGAINST HF BF16 OF THE SAME MODEL (QWEN3-32B HAS NONE: IT DOES NOT FIT THE GPU IN BF16); p IS THE TWO-SIDED EXACT MCNEMAR TEST OF THE PAIRED ANSWERS AGAINST HF BF16.

Model	Config	PPL	ΔPPL	GSM8K	Δ	p	MMLU	Δ	p
Qwen2.5-3B	HF BF16	8.56	–	85.8	–	–	63.5	–	–
	HF INT8	8.66	+0.10	85.5	−0.3	0.777	62.4	−1.1	<0.001
	HF NF4	9.11	+0.55	82.6	−3.3	<0.001	60.8	−2.8	<0.001
	vLLM BF16	8.56	0.00	86.0	+0.2	0.871	63.7	+0.1	0.075
	vLLM FP8	8.64	+0.09	85.4	−0.5	0.610	63.2	−0.4	0.038
	vLLM GPTQ-8	8.56	0.00	86.4	+0.6	0.322	63.6	+0.1	0.261
	vLLM AWQ-4	9.10	+0.54	82.7	−3.1	0.002	61.4	−2.2	<0.001
	vLLM GPTQ-4	9.22	+0.66	82.0	−3.8	<0.001	61.1	−2.5	<0.001
Qwen2.5-7B	HF BF16	7.46	–	90.8	–	–	71.4	–	–
	HF INT8	7.49	+0.03	90.6	−0.2	0.771	71.2	−0.2	0.134
	HF NF4	7.77	+0.31	90.3	−0.5	0.500	70.8	−0.6	0.002
	vLLM BF16	7.46	0.00	91.2	+0.4	0.359	71.4	0.0	0.704
	vLLM FP8	7.51	+0.05	90.8	0.0	1.000	71.3	−0.1	0.327
	vLLM GPTQ-8	7.46	0.00	91.7	+0.9	0.043	71.5	0.0	0.610
	vLLM AWQ-4	7.85	+0.39	90.4	−0.5	0.576	69.9	−1.6	<0.001
	vLLM GPTQ-4	7.78	+0.32	91.1	+0.2	0.828	70.0	−1.4	<0.001
Qwen2.5-14B	HF BF16	5.70	–	93.9	–	–	77.0	–	–
	HF INT8	5.78	+0.08	94.5	+0.6	0.280	76.6	−0.3	0.025
	HF NF4	6.07	+0.37	93.2	−0.7	0.281	76.4	−0.6	0.003
	vLLM FP8	5.77	+0.07	94.4	+0.5	0.371	76.9	0.0	0.862
	vLLM GPTQ-8	5.71	+0.01	94.7	+0.8	0.013	76.9	−0.1	0.416
	vLLM AWQ-4	6.16	+0.46	93.9	+0.1	1.000	75.1	−1.9	<0.001
	vLLM GPTQ-4	37.46‡	+31.77	93.7	−0.2	0.899	76.4	−0.6	0.005
Qwen3-8B	HF BF16	9.73	–	91.4	–	–	69.6	–	–
	HF INT8	9.72	0.00	92.0	+0.6	0.382	70.3	+0.7	<0.001
	HF NF4	9.98	+0.25	91.5	+0.2	0.909	69.3	−0.3	0.156
	vLLM BF16	9.72	0.00	92.0	+0.6	0.215	69.6	0.0	0.610
	vLLM FP8	9.73	0.00	91.8	+0.5	0.488	68.9	−0.7	<0.001
	vLLM AWQ-4	10.18	+0.46	91.1	−0.2	0.818	68.9	−0.7	0.003
Qwen3-32B	HF NF4	7.93	–	92.6	–	–	79.4	–	–
	vLLM AWQ-4	7.88§	–	94.2	–	–	78.1	–	–

‡Degraded on raw text but not on chat-formatted text; see Section IV-F. §BF16 activations: with the checkpoint's FP16 activations some 2048-token windows overflow (Section IV-H).

TABLE V

GENERALIZATION — THE HEADLINE RESULTS FOR QWEN2.5-7B AND QWEN3-8B (NON-THINKING MODE) ON THE SAME GRID ($L = 512$, 128 OUTPUT TOKENS). THROUGHPUT IS COMPARED AT THE LARGEST BATCH AT WHICH BOTH MODELS' KV CACHES HOLD EVERY REQUEST ($B = 64$). QWEN3-8B HAS NO OFFICIAL GPTQ CHECKPOINT.

Metric (vs. BF16)	Qwen2.5-7B	Qwen3-8B
Weight saving, AWQ-INT4 (vLLM)	2.70×	2.67×
Weight saving, NF4 (HF)	2.60×	2.55×
Decode speedup $B = 1$, AWQ-INT4	2.44×	2.33×
Decode speedup $B = 1$, FP8	1.59×	1.54×
Decode speedup $B = 1$, NF4	1.13×	0.90×
Decode speedup $B = 1$, LLM.int8()	0.29×	0.28×
Throughput $B = 64$, AWQ-INT4	1.15×	1.12×
Throughput $B = 64$, FP8	1.76×	1.67×
GSM8K Δ pp (p), FP8	0.0 (1.000)	+0.5 (0.488)
GSM8K Δ pp (p), LLM.int8()	−0.2 (0.771)	+0.6 (0.382)
GSM8K Δ pp (p), AWQ-INT4	−0.5 (0.576)	−0.2 (0.818)
GSM8K Δ pp (p), NF4	−0.5 (0.500)	+0.2 (0.909)
MMLU Δ pp (p), FP8	−0.1 (0.327)	−0.7 (<0.001)
MMLU Δ pp (p), LLM.int8()	−0.2 (0.134)	+0.7 (<0.001)
MMLU Δ pp (p), AWQ-INT4	−1.6 (<0.001)	−0.7 (0.003)
MMLU Δ pp (p), NF4	−0.6 (0.002)	−0.3 (0.156)

small batches of a model large enough to be memory-bound. Both remain useful to fit a model that otherwise would not fit.

(5) Prefer a larger quantized model to a smaller full-precision one. Qwen2.5-14B in AWQ-INT4 needs 10.1 GB of weights, less than the 15.2 GB of Qwen2.5-7B in BF16, and scores 93.9% vs. 90.8% on GSM8K and 75.1% vs. 71.4% on MMLU (McNemar $p < 0.001$ and $p < 0.001$).

Across all five, the bit-width alone predicts very little; the kernel that executes the format and the regime the workload puts the GPU in decide the outcome.

B. Threats to validity

One GPU, two model families. All results come from a single RTX 5090 and dense Qwen models (Qwen2.5 in three sizes; Qwen3-8B and Qwen3-32B as a check). The mechanism we identify is architectural (bytes per weight against arithmetic intensity), and Qwen3-8B reproduces it, so the qualitative picture should carry over to other dense models and GPUs, but the crossover batch sizes scale with each GPU's FLOP-to-bandwidth ratio and must be re-measured. Mixture-of-experts models, whose decode reads only a fraction of the weights, are outside our scope. *Display GPU.* The GPU also drives a desktop; the driver's watchdog aborts any kernel that runs for several seconds, which cost us one cell (LLM.int8() prefill of 16 prompts of 2048 tokens on the 14B model). *Library versions.* Kernel coverage moves fast: newer bitsandbytes releases may close part of the gap we measure, and vLLM

picks kernels per GPU architecture. We pin every version and release the harness so the study can be re-run. *Uncontrolled clocks.* We could not lock clocks without administrator rights. Repetitions agree closely (Section III), and we report means. *Synthetic prompts.* Random token ids exercise exactly the same kernels as natural text in a dense model, but they are out of distribution, which is why quality is measured separately on WikiText-2 and GSM8K and speed on real ShareGPT traffic. *Greedy decoding.* Quality numbers use greedy decoding, the single most common evaluation setting; sampling would add variance but would not change the cost of a step.

VI. Conclusion

We measured how much memory LLM quantization saves and when it makes inference faster, using eight configurations of Qwen2.5 models on an RTX 5090. Memory savings are real but smaller than advertised: 2.60× instead of 4× for 4-bit weights, and as little as 1.49× end to end once the KV cache grows. Speed depends on the kernel and the regime rather than on the bit-width: the same 8-bit weights decode 1.64× faster than BF16 with Marlin and 3.4× slower with bitsandbytes, weight-only gains disappear in the compute-bound regime, and FP8 W8A8 is the only format that is faster everywhere. 8-bit formats with good kernels preserve quality; 4-bit formats cost 0.6–2.8 MMLU points, more on smaller models, yet a larger 4-bit model still beats a smaller 16-bit one at less memory; on Qwen3-8B, however, FP8 costs as much MMLU accuracy as AWQ-INT4, so the quality ranking of formats must be measured per model. Smaller is not always faster—but with the right kernel, for the right regime, it is both smaller and faster.

Division of work

Following the proposal, Sijia Zhong built the baseline harness and the memory study (Q1), Zheng Zheng ran the controlled precision/batch/length sweeps and the kernel and profiling analysis (Q2), and Haofei Sun ran the ShareGPT and quality evaluations (Q3, Q4). All three analyzed the results and wrote the report and slides.

References

[1] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

[2] P. Micikevicius, D. Stosic, N. Burgess, M. Cornea, P. Dubey, R. Grisenthwaite, S. Ha, A. Heinecke, P. Judd, J. Kamalu, N. Mellempudi, S. Oberman, M. Shoeybi, M. Siu, and H. Wu, "FP8 formats for deep learning," *arXiv preprint arXiv:2209.05433*, 2022.

[3] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022, arXiv:2208.07339.

[4] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, arXiv:2305.14314.

[5] Hugging Face, "Bitsandbytes quantization," Transformers documentation, https://huggingface.co/docs/transformers/quantization/bitsandbytes, 2026.

[6] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," in *International Conference on Learning Representations (ICLR)*, 2023, arXiv:2210.17323.

[7] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration," in *Proceedings of Machine Learning and Systems (MLSys)*, 2024, arXiv:2306.00978.

[8] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

[9] E. Frantar, R. L. Castro, J. Chen, T. Hoefler, and D. Alistarh, "MARLIN: Mixed-precision auto-regressive parallel inference on large language models," in *Proceedings of the 30th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, 2025, pp. 239–251.

[10] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and efficient post-training quantization for large language models," in *International Conference on Machine Learning (ICML)*, 2023, arXiv:2211.10438.

[11] Qwen Team, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024.

[12] "ShareGPT_Vicuna_unfiltered (ShareGPT v3, cleaned split)," Hugging Face Datasets, https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered, 2023.

[13] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," in *International Conference on Learning Representations (ICLR)*, 2017, arXiv:1609.07843.

[14] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training verifiers to solve math word problems," *arXiv preprint arXiv:2110.14168*, 2021.

[15] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, "Measuring massive multitask language understanding," in *International Conference on Learning Representations (ICLR)*, 2021.

[16] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for transformer-based generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.

[17] Qwen Team, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.

Appendix A
Qwen2.5-3B

Figures 12 and 13 repeat the latency and crossover analysis for the 3B model. The vLLM picture is the same as for 7B. On HF the smaller model is worse off: its BF16 step is already bound by host overhead (8.3 ms per step against a 3.8 ms memory floor), so fewer bytes cannot help, NF4 never beats BF16, and LLM.int8() falls to 0.13× of BF16 speed because the 36-layer model launches more kernels per step than the 28-layer 7B model.

Appendix B
Detailed results

Tables VI and VII list the weight memory of every configuration of the 7B and 14B models and the latency, throughput and energy of the 7B model behind Figures 1 and 2.

Appendix C
Reproducibility

Every number in this report is generated from the raw measurements by `scripts/make_numbers.py`; figures and tables come from `make_figures.py` and `make_tables.py`. `run_all.sh` re-runs all experiments (about twelve GPU-hours on an RTX 5090). A job queue ran the experiments one at a time and only while no other user's process was on the GPU; rows written while the

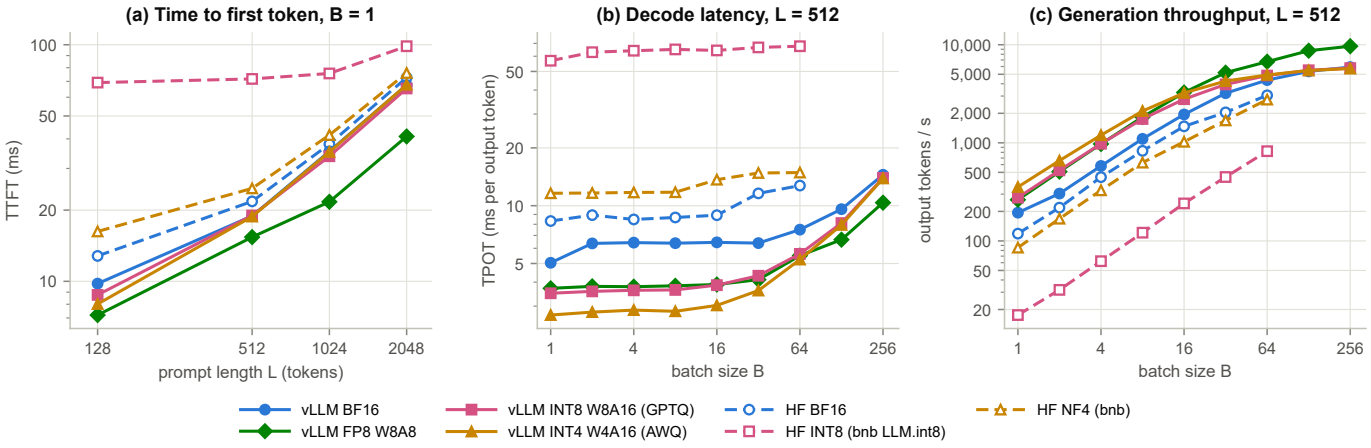


Fig. 12. Latency and throughput of Qwen2.5-3B (same layout as Figure 2).

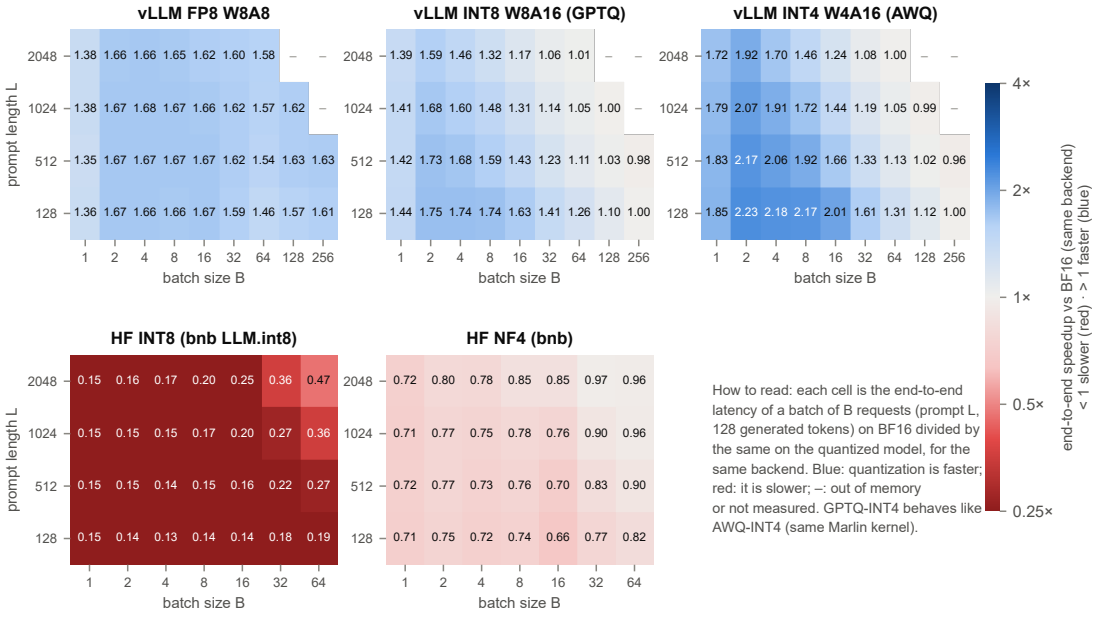


Fig. 13. End-to-end speedup over BF16 for Qwen2.5-3B (same layout as Figure 5).

GPU was shared (2 windows, 26 minutes in total, logged in `results/excluded_windows.jsonl`) are discarded and those jobs were re-run. Re-measuring vLLM BF16 on the 7B model after the interruption reproduced the original TPOT within 0.5%.

TABLE VI

Q1 — WEIGHT MEMORY: NAIVE ESTIMATE (#PARAMS×BITS/8), OUR ANALYTIC MODEL (UNQUANTIZED EMBEDDING/LM HEAD/NORMS + QUANTIZATION METADATA), AND MEASURED ON THE GPU.

Model	Config	Naive (GB)	Model (GB)	Measured (GB)	Ratio vs BF16	Naive ratio
Qwen2.5-7B	HF BF16	15.23	15.23	15.23	1.00×	1.00×
Qwen2.5-7B	HF INT8 (LLM.int8)	7.62	8.71	8.71	1.75×	2.00×
Qwen2.5-7B	HF NF4	3.81	5.85	5.85	2.60×	4.00×
Qwen2.5-7B	vLLM BF16	15.23	15.23	15.34	1.00×	1.00×
Qwen2.5-7B	vLLM FP8 W8A8	7.62	8.71	8.77	1.75×	2.00×
Qwen2.5-7B	vLLM GPTQ W8A16	7.62	8.86	8.90	1.72×	2.00×
Qwen2.5-7B	vLLM AWQ W4A16	3.81	5.57	5.68	2.70×	4.00×
Qwen2.5-7B	vLLM GPTQ W4A16	3.81	5.57	5.66	2.71×	4.00×
Qwen2.5-14B	HF BF16	29.54	29.54	29.54	1.00×	1.00×
Qwen2.5-14B	HF INT8 (LLM.int8)	14.77	16.33	16.34	1.81×	2.00×
Qwen2.5-14B	HF NF4	7.39	10.55	10.58	2.79×	4.00×
Qwen2.5-14B	vLLM BF16	29.54	29.54	29.60	1.00×	1.00×
Qwen2.5-14B	vLLM FP8 W8A8	14.77	16.33	16.51	1.79×	2.00×
Qwen2.5-14B	vLLM GPTQ W8A16	14.77	16.64	16.73	1.77×	2.00×
Qwen2.5-14B	vLLM AWQ W4A16	7.39	9.98	10.07	2.94×	4.00×
Qwen2.5-14B	vLLM GPTQ W4A16	7.39	9.98	10.02	2.95×	4.00×

TABLE VII

Q2 — LATENCY, THROUGHPUT AND ENERGY FOR QWEN2.5-7B AT PROMPT LENGTH 512 AND 128 OUTPUT TOKENS (MEAN OF 3 RUNS). SPEEDUP IS RELATIVE TO BF16 ON THE SAME BACKEND.

Config	TTFT B=1 (ms)	TPOT B=1 (ms)	TPOT B=64 (ms)	tok/s B=64	Decode speedup B=1	mJ/token B=1	mJ/token B=64
HF BF16	38.4	10.90	20.04	1669	1.00×	5092	294
HF INT8 (LLM.int8)	57.8	37.45	47.14	1003	0.29×	6187	305
HF NF4	48.3	9.62	22.87	1550	1.13×	3188	340
vLLM BF16	37.0	9.52	12.39	2209	1.00×	4878	249
vLLM FP8 W8A8	22.8	5.99	7.88	3886	1.59×	2507	141
vLLM GPTQ W8A16	38.5	5.80	8.41	2471	1.64×	2588	234
vLLM AWQ W4A16	38.6	3.90	7.99	2544	2.44×	1919	224
vLLM GPTQ W4A16	36.9	3.88	7.89	2642	2.45×	1840	219